

Exhausted or Nauseous

Full Curriculum Pack

Build a Social Status App with Expo 56 & Supabase

Source: this repository · Pack v1.0 · August 2026

Course: Build a Social Status App with Expo 56 & Supabase

Product: Exhausted or Nauseous (EON) — “Tell friends how you feel”

Source of truth: this repository

Expo docs: <https://docs.expo.dev/versions/v56.0.0/>

Estimated runtime: ~11 hours of lecture/lab · ~16 days at a steady pace

Pack version: 1.0 · August 2026

How to use this pack

This is a full curriculum pack, not a video course. Use it in one of three modes:

- Rebuild from scratch — Create a new Expo 56 app and follow Sections 2–9 in order. Use this repo as the answer key when stuck.
- Reverse-engineer this repo — Stay in the existing project. For each lecture, open the listed files and walk the flow.
- Teach / mentor — Use Teaching notes + Labs + Checkpoints. Capstone rubric is in Section 9.

Per-lecture loop

- Read outcomes and concepts
- Do the lab (or read the key files)
- Hit the checkpoint
- Skim pitfalls before moving on

Companion: the interactive syllabus canvas in Cursor ([lesson-plan.canvas.tsx](#)) mirrors section/lecture IDs used here (e.g. 4.2).

Part A — Course overview

What you will ship

A tiny social status app:

- Sign in with phone OTP (SMS)
- Complete onboarding (name, username, optional photo)
- Tap Exhausted or Nauseous so friends see your vibe
- Search / request / accept friends
- Get in-app + push notifications when friends check in

Intentionally out of scope: payments, feeds, DMs, analytics SDKs, multi-region phone UX beyond US +1.

Learning outcomes

By the end you can:

- Scaffold an Expo 56 + TypeScript + expo-router app with brand-first UI
- Wire Supabase Auth (phone OTP), Postgres, RLS, Storage, and Edge Functions
- Model a friend graph and optimistic status updates
- Fan out notifications with Postgres triggers (not N client inserts)
- Deliver Expo Push via a Deno edge function + DB webhook/pg_net
- Harden PII with column-level grants
- Package with EAS and prepare App Store Connect artifacts

Stack (exact versions in this repo)

- Layer · Choice
- Expo · ~56.0.12
- expo-router · ~56.2.11
- React / RN · 19.2.3 / 0.85.3
- Supabase JS · ^2.108.2
- TypeScript · ~6.0.3
- AsyncStorage · 2.2.0
- Reanimated · 4.3.1
- Notifications · expo-notifications ~56.0.20
- Image picker · expo-image-picker ~56.0.20

App id: com.ndoelgersteam.exhaustedornauseous · scheme: exhaustedornauseous

Architecture at a glance

```
Mobile (Expo)
├─ Auth gate (src/app/index.tsx)
├─ Screens: login, onboarding, home
├─ Modals: profile, friends, search, notifications
└─ utils/supabase.ts → Supabase
```

```
├─ Auth (phone OTP)
├─ Postgres (profiles, friend_requests, notifications)
├─ Storage (avatars)
└─ Edge Function push ← DB trigger / pg_net
    └─ Expo Push API → friend devices
```

Golden demo path: OTP → onboard → become friends → tap emotion → friend sees notification (and push on a real device).

Part B — Prerequisites & setup

Student prerequisites

- Node 20+
- Xcode (iOS) and/or Android Studio
- Expo account (for EAS / push projectId)
- Free Supabase project
- Comfortable with TypeScript and basic SQL

Environment variables

From `.env.example`:

- Variable · Required · Role
- EXPO_PUBLIC_SUPABASE_URL · App · Project URL
- EXPO_PUBLIC_SUPABASE_KEY · App · Anon key
- SUPABASE_SERVICE_ROLE_KEY · Scripts only · Seed / admin

Missing public vars throw in `utils/supabase.ts`.

Push pipeline extras (Section 7):

- Edge: SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY (auto in Deno)
- Optional: EXPO_ACCESS_TOKEN
- DB: public.app_config row key='supabase_url'

Essential npm scripts

- Script · Purpose
- npm start · Expo dev server
- npm run ios / android · Native run
- npm run supabase:start · Local Supabase
- npm run db:reset · Reset + apply migrations
- npm run db:push:develop · Push migrations to develop branch
- npm run seed:dev-users · Seed (legacy email-era — see pitfalls)
- npm run test:friends · Friends integration script

Also: eas build, eas submit, supabase functions deploy push.

Setup lab (do once)

- Clone repo · npm install
- Copy `.env.example` → `.env` · fill Supabase URL + anon key
- Enable Phone auth in Supabase (SMS provider)
- npx supabase start (optional local) or point at cloud
- npm start · open iOS simulator

Checkpoint: App launches; without a session you see Login.

Part C — 16-day pacing guide

- Days · Focus · Sections
- 1–2 · Vision, Expo scaffold, static Home · 1–2
- 3–4 · Supabase client, profiles, RLS, CLI · 3
- 5–7 · OTP, auth gate, onboarding, avatars · 4
- 8 · Persist emotion + optimistic UI · 5
- 9–11 · Friend requests, search, modals · 6
- 12–14 · Triggers, in-app list, Edge push · 7
- 15–16 · Security review + store submit · 8–9

Adjust: reverse-engineering mode can compress Days 1–4 into one afternoon.

Part D — Curriculum (Sections 1–9)

Section 1 — Welcome & Product Vision

~30 min · Beginner

1.1 What is Exhausted or Nauseous? (8 min)

Outcomes

- Explain the Yo-style binary status concept
- List core loops: check-in, friends, notify
- Know what is out of scope

Concepts

The product is deliberate minimalism: two emotions, no feed, no endless scroll. Friends see how you are via a single tap. Brand voice is short (“we let them know.”). Store copy lives in `store.config.json`.

Lab

- Read `store.config.json` → `apple.info.en-US` description
- Open `docs/index.html` marketing page
- Write three bullets: who it’s for, core action, what you will not build

Checkpoint: You can pitch the app in 20 seconds without mentioning a feed.

Key files: `store.config.json`, `docs/index.html`

1.2 Stack overview (12 min)

Outcomes

- Name each architecture layer
- Treat Expo v56 docs as API source of truth
- Sketch emotion → push path

Concepts

Client talks directly to Supabase (no custom BFF). Side effects for social fan-out live in Postgres triggers. Push is decoupled: insert notification → forward to Edge Function → Expo Push API.

Lab

Draw the diagram from Part A on paper. Label where RLS runs, where the edge function runs, and where the mobile app stops being responsible.

Checkpoint: You can explain why the client does not insert N notification rows itself.

1.3 Repo map & demo path (10 min)

Outcomes

- Locate screens vs helpers vs migrations
- Describe the golden demo

Repo map

- Path · Role
- src/app/ · Screens + auth gate
- src/components/ · Forms + modals
- utils/ · supabase, friends, register-push
- supabase/migrations/ · Schema evolution
- supabase/functions/push/ · Push sender
- assets/emotions/ · Big tap images
- store/ · ASC checklist, privacy, screenshots
- scripts/ · seed, friends test, db push

Lab: From repo root, list those folders and open src/app/index.tsx once — do not deep-read yet.

Checkpoint: You know where to look for UI vs SQL vs push.

Section 2 — Expo Scaffold & Brand UI

~75 min · Beginner

2.1 Create an Expo 56 TypeScript app (20 min)

Outcomes: Run on simulator · explain expo-router file routing

Concepts: main: expo-router/entry · app code under src/app/ · @/* → ./src/* in tsconfig.json · experiments in app.json: typedRoutes, reactCompiler.

Lab (rebuild mode)

- npx create-expo-app@latest targeting SDK 56
- Enable expo-router · move routes under src/app
- Mirror path alias from this repo's tsconfig.json

Key files: package.json, tsconfig.json, app.json

Checkpoint: npm start shows a blank routed screen without redbox.

Pitfall: Mixing SDK versions — AGENTS.md requires v56 docs only.

2.2 Root layout, splash, fonts (18 min)

Outcomes: Prevent FOUT · wire splash correctly

Concepts: `_layout.tsx` loads Google fonts via `useFonts`, keeps splash visible until ready, wraps `SafeAreaProvider` + headerless Stack.

Lab: Read `src/app/_layout.tsx` and `src/fonts.ts`. Note default pack is DM Sans even though many font packages are installed.

Key files: `src/app/_layout.tsx`, `src/fonts.ts`

Checkpoint: Cold start shows splash color `#1B2A4A` then UI with DM Sans.

2.3 Theme tokens & brand colors (12 min)

Outcomes: Centralize color/type

Brand tokens (`src/theme.ts`)

- Navy: `#1B2A4A`, `#152238`, `#0F1A2E`
- Accent green: `#009c4e` (pressed emotion / badge)
- Neutrals: black `#111`, cream `#F0F3F8`, muted `#5C6570`
- Danger: `#C0392B`
- Type scale: hero 64 / title 28 / body 16 / meta 14

Lab: Recreate colors + type objects. Reference them from a throwaway Text.

Checkpoint: No hardcoded navy hex scattered in new code you write.

2.4 Static Home: two emotion buttons (25 min)

Outcomes: Ship brand-first first viewport · prove UX before backend

Concepts: Home is the product: two large circular emotion images (`assets/emotions/`), ME / FRIENDS / ACTIVITY chrome, local `useState` only at this stage.

Lab

- Study `src/app/home.tsx` layout structure
- In rebuild mode, implement static buttons + press feedback without Supabase
- Optional: Reanimated confirmation slot (“we let them know.”)

Key files: `src/app/home.tsx`, `assets/emotions/`

Checkpoint: First viewport reads as one composition: brand + two taps — not a dashboard.

Section 3 — Supabase Foundation

~105 min · Intermediate

3.1 Create Supabase project & env (15 min)

Lab: Create project · copy URL + anon key into .env · never commit service role to the app bundle.

Checkpoint: .env present locally; .env.example documents required keys.

3.2 Supabase JS client + AsyncStorage session (20 min)

Outcomes: Persist auth across restarts · single shared client

Concepts (utils/supabase.ts): createClient with persistSession, autoRefreshToken, detectSessionInUrl: false, AsyncStorage adapter.

Lab: Implement or read the client module. Import it from a temporary screen and console.log supabase.auth.getSession().

Key files: utils/supabase.ts

Checkpoint: Killing the app keeps the session (after auth exists in Section 4).

3.3 profiles table + handle_new_user (30 min)

Outcomes: Write auth → profile trigger · run db reset

Concepts: On auth.users insert, trigger creates a profiles row. Later migrations changed this from email-based to phone stub with nullable names (...profiles_onboarding_stub).

Lab

- Read earliest migration 20260629220000_create_profiles.sql
- Read 20260802183000_profiles_phone_replace_email.sql and onboarding stub migration
- Sketch columns: id, phone, names, username, emotion, avatar_url, expo_push_token

Key files: supabase/migrations/ (profiles family)

Checkpoint: New auth user ⇒ profile row exists (test after Section 4 OTP).

3.4 Row Level Security basics (25 min)

Outcomes: RLS on before shipping · own-row writes · searchable selects

Concepts: Authenticated users can SELECT profiles for search; UPDATE/INSERT restricted to own id. Later, column grants hide phone and expo_push_token even when SELECT is allowed (Section 8).

Lab: Read ...profiles_search_policy.sql and grant migrations. Write the policy intent in plain English.

Checkpoint: You can explain “searchable but not writable by others.”

3.5 Local Supabase CLI workflow (15 min)

Lab: supabase start · npm run db:reset · skim scripts/db-push-develop.js for branch-aware push.

Key files: supabase/config.toml, scripts/db-push-develop.js

Checkpoint: Migrations apply cleanly on a fresh reset.

Section 4 — Phone Auth & Onboarding

~145 min · Intermediate / Advanced

4.1 Enable phone auth (12 min)

Lab: Supabase dashboard → Auth → Phone · configure SMS. Understand US +1 formatting used by the app.

Pitfall: Login UI hardcodes 10-digit US numbers.

4.2 LoginForm: signInWithOtp (22 min)

Outcomes: Validate phone · send OTP · handle errors

Flow: 10 digits → +1XXXXXXXXXX → supabase.auth.signInWithOtp({ phone }).

Key files: src/components/login-form.tsx, src/app/login.tsx

Lab: Trace the submit handler. In rebuild mode, recreate the form with the same validation rules.

Checkpoint: Submitting a valid number triggers SMS (or Supabase test OTP in local/dev).

4.3 VerifyOtpForm (20 min)

Flow: 6-digit code → verifyOtp({ phone, token, type: "sms" }) → session.

Key files: src/components/verify-otp-form.tsx

Checkpoint: Verified session appears in getSession().

4.4 Auth gate in index.tsx (28 min)

Outcomes: Prefer conditional render for this app size · load profile after session

Flow

```
getSession / onAuthStateChange
  no session → Login
  session → loadProfile(id, session.user.phone)
    fail → signOut → Login
    ok → registerPushToken
      needsOnboarding? → Onboarding : Home
```

Important: Phone for the Profile object comes from `session.user.phone`, not from `selecting profiles.phone` (hidden later).

Key files: `src/app/index.tsx`, `src/types/profile.ts`

Lab: Read `needsOnboarding` logic and `mapProfile`.

Checkpoint: Fresh OTP user lands on Onboarding, not Home.

4.5 Onboarding: name, username, stub profile (25 min)

Outcomes: `snake_case` ↔ `camelCase` mapping · unique usernames

Concepts: Stub profile has null names. Onboarding upserts `first_name`, `last_name`, `username` (+ optional avatar). Types live in `src/types/profile.ts` (`mapProfile`, `unwrapProfile`, `EMOTION` constants).

Key files: `src/app/onboarding.tsx`, `src/types/profile.ts`

Checkpoint: After save, gate opens Home.

4.6 Avatar upload — ImagePicker + Storage (30 min)

Outcomes: Storage policies · RN upload quirks

Concepts

- Bucket: `avatars` (public read)
- Path: `user-{uuid}/profile.{ext}`
- Upload via `arrayBuffer()` (not `Blob`) — called out in `profile/onboarding` comments
- Cache-bust public URL with `?t=Date.now()`

Key files: `src/components/profile-modal.tsx`, `src/app/onboarding.tsx`, `migration ...avatars_storage.sql`

Lab: Complete one avatar change from ME modal; confirm `Storage` object + `avatar_url` update.

Checkpoint: Avatar visible on Home / friends list after refresh.

Pitfall: Using `Blob` uploads on RN often fails silently or oddly — prefer `ArrayBuffer`.

4.7 Aside: email signup was earlier (8 min)

Read: `src/components/signup-form.tsx` (legacy). Migration `...phone_replace_email` drops email. Seed scripts may still be email-era.

Checkpoint: You will not wire email signup into Login.

Section 5 — Core Emotion Loop

~52 min · Beginner / Intermediate

5.1 emotion column (12 min)

Stored values: "Exhausted" | "Nauseous" | null

App mapping: EMOTION + mapEmotion in src/types/profile.ts

Lab: Confirm migration ...profiles_add_emotion.sql and later emoji value migration.

5.2 Optimistic updateEmotion (25 min)

Pattern (index.tsx)

- Map button key → emoji string
- setProfile optimistically
- profiles.update({ emotion }).eq("id", ...)
- On error: restore previous · Alert

Lab: Trace updateEmotion and home.tsx handleEmotion. In rebuild mode, implement the same rollback behavior.

Checkpoint: Airplane mode → tap → UI reverts + alert.

5.3 Confirmation copy & motion (15 min)

Show “we let them know.” in a fixed-height slot ~1s, then fade. Green press state while finger down. Prefer Reanimated for enter/exit of modals (animated-modal.tsx).

Checkpoint: Confirmation never pushes layout around chaotically.

Section 6 — Friends Graph

~127 min · Intermediate / Advanced

6.1 friend_requests schema (20 min)

Table: from_user, to_user, status (pending / accepted / rejected), uniqueness, RLS, delete policy for either party.

Key migrations: ...friend_requests.sql, ...friend_requests_delete.sql

Checkpoint: You can draw the state machine for a request.

6.2 Friends helper API (25 min)

File: `utils/friends.ts`

- Export · Role
- `listFriends` · Accepted + pending incoming
- `getFriendship` · Status relative to me
- `sendFriendRequest` · Insert pending
- `respondFriendRequest` · accept / reject
- `unfriend` · Delete row

FriendStatus: none | pending_sent | pending_received | accepted | rejected | self

PROFILE_FIELDS for joins: id, username, names, avatar_url, emotion

Lab: Read each export; write a one-line summary in your notes.

6.3 FIND FRIENDS search (22 min)

`src/components/search.tsx` — `ilike` on `username` / `first` / `last` · opens `UserProfileModal`.

Checkpoint: Search finds a seeded or second-device user.

6.4 UserProfileModal actions (20 min)

UI driven by `getFriendship` status: `send` / `accept` / `reject` / `unfriend`.

Key file: `src/components/user-profile-modal.tsx`

6.5 FriendsModal (22 min)

Pending YES/NO + accepted list with their emotion. Friend request UX lives here (not in `ACTIVITY`).

Key file: `src/components/friends-modal.tsx`

Pitfall: `ACTIVITY` badge/modal filters `type = 'emotion'` only.

6.6 Lab: seed & test scripts (18 min)

`npm run seed:dev-users` · `npm run test:friends`

Note: scripts may still assume email-era auth — adapt or use two real phone users.

Checkpoint: Accept a request; both sides see friendship + emotion.

Section 7 — Notifications: In-App & Push

~135 min · Intermediate / Advanced

7.1 notifications table (15 min)

Recipient-owned rows: actor, type, body, read_at, RLS so only recipient reads/updates.

Types include: emotion check-ins, friend_request, friend_accepted.

7.2 Trigger: notify friends on emotion (25 min)

Idea: AFTER UPDATE on profiles.emotion → insert notification rows for accepted friends.

Evolution pitfall: Early trigger used IS DISTINCT FROM old.emotion (re-taps silent). Later fix (...fix_push_notifications) notifies whenever emotion IS NOT NULL so same-emotion re-taps still ping friends.

Lab: Read ...emotion_notifications.sql then the fix migration. Diff the condition in your notes.

7.3 Friend request / accepted triggers (18 min)

Migrations: ...friend_request_notifications.sql, ...friend_accepted_notifications.sql (+ copy tweaks).

Checkpoint: Sending/accepting a request creates the expected notification rows.

7.4 In-app NotificationsModal (20 min)

src/components/notifications-modal.tsx — emotion notifications; marks unread read_at on open. Home realtime unread badge on notifications.

Lab: Tap emotion on device A; open ACTIVITY on device B.

7.5 Register Expo push token (22 min)

utils/register-push.ts:

- Notification handler
- Android channel
- Permissions
- getExpoPushTokenAsync with EAS projectId
- profiles.update({ expo_push_token }) with .select("id") only

Pitfalls

- After column revoke, do not .select("expo_push_token")

- Simulators often fail token registration — use a physical device

Called from index.tsx after profile load.

7.6 Edge Function push + webhook (35 min)

File: supabase/functions/push/index.ts

Pipeline

- Trigger inserts notifications
- forward_notification_to_push reads app_config.supabase_url · pg_net HTTP POST to /functions/v1/push
- Function handles INSERT payload
- RPC get_expo_push_token(target_user) (security definer)
- POST Expo Push API · title “Exhausted or Nauseous”

Config checklist

- Deploy function
- Insert app_config supabase_url
- Optional EXPO_ACCESS_TOKEN
- Service role can read tokens via RPC

Pitfall: Hiding columns without RPC broke push (“permission denied for table profiles”) — fixed in ...fix_push_notifications.sql.

Checkpoint: Real device receives a push when a friend taps an emotion.

Section 8 — Security Hardening

~52 min · Intermediate / Advanced

8.1 Threat model (12 min)

Enumerate: leaked phones, stolen push tokens, spoofed emotions, friend spam, avatar overwrite.

Lab: Rank top three for this app size.

8.2 Column-level grants (25 min)

Migration ...profiles_hide_sensitive_columns.sql:

- Clients cannot SELECT phone or expo_push_token
- Profiles remain searchable on public columns
- Own phone from auth session
- Push reads token via security-definer RPC

Lab: Attempt selecting phone from the client — confirm failure. Confirm public fields still work.

8.3 Avatar & Storage policy review (15 min)

Public read · write only under own user-`{id}/` prefix · prevent cross-user overwrite.

Checkpoint: Policy review checklist written (who can read/write what).

Section 9 — Polish & Ship

~118 min · Mixed

9.1 EAS profiles (20 min)

eas.json: development (dev client), preview, production (+ simulator variants). Know when you need a dev client vs store build.

Lab: eas build --profile preview (optional; takes time).

9.2 Store metadata & privacy labels (18 min)

- store.config.json — title, subtitle, description, keywords, review notes
- store/APP_PRIVACY.md — Phone, Name, User ID, Photos, Other User Content, Device ID — functionality, linked, no tracking

Checkpoint: Declared data matches real behavior.

9.3 Marketing / privacy / support pages (15 min)

docs/index.html, privacy.html, support.html — host via GitHub Pages for ASC URL fields.

9.4 Screenshots, checklist, submit (25 min)

Follow store/ASC_CHECKLIST.md:

- Screenshots 6.7/ (1290×2796) and 6.1/ (1179×2556)
 - Icon 1024
 - Age rating / no kids / encryption flag
 - eas metadata:lint → eas metadata:push
 - Screenshots often still manual in ASC
-

9.5 Capstone: golden demo (40 min)

Rubric (pass = all required)

- # · Criterion · Req
- 1 · Two accounts complete phone OTP · Required
- 2 · Both finish onboarding (names + username) · Required
- 3 · Friend request sent and accepted · Required
- 4 · Emotion tap updates own UI immediately · Required
- 5 · Friend sees in-app emotion notification · Required
- 6 · Push received on physical device (if tokens configured) · Stretch
- 7 · Avatar optional but if set, visible to friend · Stretch
- 8 · Sign out returns to Login; session cleared · Required
- 9 · Short retro: one thing you'd cut next time · Required

Retro prompt: What would you delete to ship a week earlier without killing the golden demo?

Part E — Appendices

Appendix A — Screen & component catalog

Screens (src/app/)

- File · Role
- _layout.tsx · Fonts, splash, Stack
- index.tsx · Auth gate, profile, updateEmotion, push registration
- login.tsx · Brand + OTP shell
- onboarding.tsx · Profile completion + optional avatar
- home.tsx · Emotion taps + modal chrome + realtime badge

Components (src/components/)

- File · Role
 - login-form.tsx · Send SMS OTP
 - verify-otp-form.tsx · Verify code
 - signup-form.tsx · Legacy email (unused)
 - avatar.tsx · Photo or initials
 - animated-modal.tsx · Reanimated modal shell
 - profile-modal.tsx · Edit self, avatar, sign out
 - user-profile-modal.tsx · Other user + friendship actions
 - friends-modal.tsx · Requests + friends list
 - notifications-modal.tsx · Emotion activity + mark read
 - search.tsx · Find friends overlay
 - font-picker.tsx · Dev-only font comparer
-

Appendix B — Migration timeline (teaching order)

- Migration · Purpose
- ...create_profiles · profiles + handle_new_user
- ...grant_profiles · CRUD grants
- ...fix_profiles_email · Early email alignment
- ...profiles_search_policy · Searchable SELECT RLS
- ...profiles_add_emotion · emotion column
- ...avatars_storage · avatar_url + Storage
- ...friend_requests · friend graph
- ...friend_requests_delete · either-party delete
- ...emotion_notifications · notifications + emotion trigger
- ...expo_push_tokens · token column + richer body
- ...notification_body_emoji · emoji in body
- ...friend_request_notifications · type friend_request
- ...friend_accepted_notifications · type friend_accepted

- ...friend_notification_copy · copy tweak
 - ...emotion_column_emoji · store emoji in emotion values
 - ...profiles_phone_replace_email · phone replaces email
 - ...profiles_onboarding_stub · nullable stub on OTP
 - ...profiles_hide_sensitive_columns · hide phone + push token
 - ...fix_push_notifications · RPC, re-tap notify, pg_net forward, realtime
-

Appendix C — Push pipeline (ops checklist)

- registerPushToken succeeds on a physical device
- app_config.supabase_url set
- Edge function push deployed
- Trigger/pg_net forwards on notification INSERT
- get_expo_push_token RPC works under service role
- Optional Expo access token for enhanced security
- Test: friend tap → Expo receipt / device banner

If in-app works but push does not: check token registration, app_config, function logs, and column/RPC permissions — not the Home UI.

Appendix D — Profile type cheat sheet

```
Emotion = "Exhausted" | "Nauseous" | null
```

```
Profile (app): id, phone, firstName, lastName, username, emotion, avatarUrl
```

```
ProfileRow (db): id, phone?, first_name?, last_name?, username?, emotion?, avatar_url?
```

```
Public SELECT columns: id, username, first_name, last_name, avatar_url, emotion
```

```
Phone: session.user.phone only
```

Helpers: mapEmotion, mapProfile, unwrapProfile

Appendix E — Pitfalls catalog

- Email → phone: Don't follow signup-form / email seed scripts as current auth.
- Hidden columns: Never SELECT phone / expo_push_token from the client after grants revoke.
- Push + grants: Need security-definer RPC for token reads.
- ArrayBuffer uploads for Storage on RN.
- Same-emotion re-taps must still notify (fixed trigger condition).
- registerPushToken select: .select("id") only.
- ACTIVITY vs FRIENDS: emotion notifications vs request UI.
- Stub onboarding required after OTP.

- Missing app_config skips device push; in-app still works.
 - US +1 only in LoginForm.
 - Physical device for reliable Expo push tokens.
 - Many fonts loaded, one default — don't ship font experiments by accident.
-

Appendix F — Glossary

- Term · Meaning
 - Auth gate · Conditional render Login / Onboarding / Home from session+profile
 - Stub profile · Row created at OTP with null names
 - Optimistic update · UI updates before server confirms; roll back on error
 - Fan-out · Creating many notification rows from one emotion change
 - RLS · Postgres Row Level Security policies
 - Column grants · Privilege layer hiding specific columns
 - EAS · Expo Application Services (build/submit)
 - ASC · App Store Connect
 - Golden demo · OTP → onboard → friends → tap → notify
-

Appendix G — Instructor tips

- Demo with two phones from Section 6 onward.
 - When students get stuck on push, isolate: DB row created? → function invoked? → Expo accepted? → device token valid?
 - Encourage reading migrations in chronological order — the schema story is the course story.
 - Scope discipline is a feature: praise deleting ideas that aren't in the golden demo.
 - Always point API questions at <https://docs.expo.dev/versions/v56.0.0/>
-

Part F — Detailed lab walkthroughs

Use these when a lecture's short Lab block is not enough. Each walkthrough assumes reverse-engineer mode in this repo unless noted.

Lab W1 — Trace a cold start (Sections 2 + 4)

Goal: Explain every screen decision from launch to Home.

- Open `src/app/_layout.tsx`. Note splash hold until fonts load.
- Open `src/app/index.tsx`. Find `getSession` and `onAuthStateChange`.
- With no session, confirm render path returns `<Login />`.
- After OTP (or a saved session), follow `loadProfile`:
 - Which columns are selected?
 - Where does phone come from?
 - When is `registerPushToken` called?
- Force onboarding by clearing names in Supabase Table Editor; relaunch; confirm gate.
- Complete onboarding; confirm Home mounts with `updateEmotion` prop.

Write-up (5 sentences): Session source, profile source, onboarding predicate, push registration timing, emotion update ownership.

Lab W2 — Rebuild Login OTP without peeking (Section 4)

Goal: Prove you understand Auth APIs.

- Create a throwaway screen or branch.
- Implement phone field (10 digits) + Send.
- Call `signInWithOtp({ phone: "+1..." })`.
- Implement verify with `verifyOtp({ type: "sms" })`.
- Only then diff against `login-form.tsx` / `verify-otp-form.tsx`.

Success: Your version handles empty input, non-10-digit input, and surfaces Supabase errors.

Lab W3 — Emotion optimistic rollback drill (Section 5)

- Sign in on a simulator.
- Enable Network Link Conditioner / airplane mode after Home loads.
- Tap Exhausted. Observe optimistic UI.
- Wait for failure path: previous emotion restored + alert.
- Re-enable network; tap again; confirm persistence in Supabase Table Editor.

Success: You can point to the exact prev restore lines in `index.tsx`.

Lab W4 — Friendship state machine (Section 6)

Draw a table with columns: Action, from_user, to_user, status before, status after.

Execute with two accounts:

- A searches B → send request
- B opens Friends → pending → Accept
- A opens B's profile → Unfriend
- Optional: reject path instead of accept

For each step, run getFriendship mentally (or log it) and fill the table.

Success: No duplicate pending rows; unfriend deletes the row; rejected state behaves as coded.

Lab W5 — Notification fan-out SQL reading (Section 7)

- Open ...emotion_notifications.sql and the later fix migration.
- Copy the WHEN / IF condition from each into your notes.
- Explain in one paragraph why re-tapping the same emotion should notify.
- Insert a manual friendship + update emotion in SQL editor; count notifications rows.

Success: Row count equals number of accepted friends (for that actor).

Lab W6 — Push isolation ladder (Section 7.6)

When push fails, climb this ladder in order — do not skip:

- Does notifications row exist for the recipient?
- Is app_config.supabase_url set?
- Do Edge Function logs show an invoke?
- Does get_expo_push_token return a token for that user?
- Does Expo push tool / receipts show success?
- Is the device a physical device with notification permission?

Document which rung failed on your first attempt.

Lab W7 — Sensitive column audit (Section 8)

From the app client (or a scratch script using the anon key + user JWT):

- select * from profiles — list returned keys.
- Confirm phone and expo_push_token absent.
- Confirm session.user.phone still available.
- Update emotion and avatar; confirm still works.

- Attempt to update another user's emotion; confirm RLS blocks.

Success: Written audit with pass/fail per step.

Lab W8 — Store packet dry run (Section 9)

Without submitting:

- Fill a checklist copy of store/ASC_CHECKLIST.md.
- Open store.config.json and verify privacy/support/marketing URLs resolve.
- Confirm screenshot folders exist with expected sizes.
- Read APP_PRIVACY.md and map each data type to a code path in this repo.

Success: Mapping table with file references for each privacy nutrition category.

Part G — Discussion prompts & quiz bank

Discussion prompts (use in group or journaling)

- Why is conditional render acceptable here but dangerous in a 40-screen app?
- What product features would force you to add a BFF?
- Is searchable-by-all-authenticated-users the right privacy default?
- Should re-tapping the same emotion notify friends? Argue both sides.
- What would you monitor in production first: auth failures, push failures, or friend spam?

Quiz — mark true/false (answers below)

- Home uses expo-router tabs for ME / FRIENDS / ACTIVITY.
- Own phone is loaded from profiles.phone SELECT.
- Emotion updates are optimistic with rollback.
- Friend request notifications appear in the ACTIVITY modal.
- Push delivery is performed inside the React Native app process.
- Column grants alone replace the need for RLS.
- Avatar uploads should prefer ArrayBuffer on React Native.
- Expo v55 docs are fine for this project.
- registerPushToken should .select("expo_push_token") after update.
- Capstone requires two accounts and an accepted friendship.

Answers: 1 F (modals + conditional gate) · 2 F (session.user.phone) · 3 T · 4 F (FRIENDS modal; ACTIVITY is emotion) · 5 F (Edge Function + Expo API) · 6 F · 7 T · 8 F (v56 only) · 9 F (select id only) · 10 T

Part H — Day-by-day instructor agenda

Day 1 — Vision + scaffold

- 20 min product pitch + out-of-scope list
- 40 min Expo 56 project tour / create
- 30 min theme + static Home lab
- Exit ticket: screenshot of static Home

Day 2 — Supabase foundation

- 15 min project/env
- 45 min profiles + trigger reading
- 30 min RLS workshop
- 20 min CLI reset drill

Day 3 — Auth

- 60 min OTP forms live code
- 40 min auth gate walkthrough
- Exit ticket: session persists after reload

Day 4 — Onboarding + avatars

- 40 min onboarding fields + uniqueness
- 50 min Storage upload lab
- 20 min legacy email aside + Q&A

Day 5 — Emotion loop

- 30 min types + DB values
- 45 min optimistic update pair programming
- 20 min motion polish

Day 6 — Friends

- 40 min schema + helpers
- 50 min search + modals with two devices
- 20 min state machine worksheet (Lab W4)

Day 7 — Notifications in-app

- 30 min table + triggers
- 40 min re-tap condition debate + SQL
- 30 min ACTIVITY modal + realtime badge

Day 8 — Push

- 30 min token registration
- 60 min edge function deploy + ladder (Lab W6)
- 20 min failure postmortem shares

Day 9 — Security + ship prep

- 40 min column grant audit (Lab W7)
- 40 min store packet dry run (Lab W8)
- 30 min capstone planning

Day 10 — Capstone

- 90 min golden demo attempts
- 30 min rubric scoring
- 20 min retros

(Compress to 5 days by pairing Days 1–2, 3–4, 5–6, 7–8, 9–10.)

Part I — Troubleshooting playbook

- Symptom · Likely cause · Where to look
 - Stuck on Login after OTP · Profile load failed / RLS · index.tsx loadProfile, grants
 - Onboarding loops · Username unique violation / save error · onboarding.tsx, DB constraints
 - Emotion tap no friend ping · Not friends / old trigger condition · friend_requests, emotion trigger SQL
 - In-app works, no push · token / app_config / function · Lab W6 ladder
 - Avatar blank for others · URL/cache / storage policy · avatar_url, bucket policies
 - “permission denied for table profiles” in function · service role / RPC missing · ...fix_push_notifications
 - Seed script auth fails · email-era script vs phone auth · scripts/seed-dev-users.js
 - Push only on one device · simulator token failure · physical device, permissions
-

Part J — Suggested extension projects (post-capstone)

Only after the golden demo passes:

- International phone input (drop hardcoded +1)
- Block / report user
- Mute notifications per friend
- Emotion history (still no feed — maybe last 5)
- Replace modal stack with explicit routes
- Add Detox/Maestro smoke test for OTP→Home

Each extension should include a threat-model note and an RLS plan before coding.

End of curriculum pack

Built from the Exhausted or Nauseous repository syllabus (Sections 1–9, 43 lectures).

Includes overview, setup, pacing, full lecture sheets, labs W1–W8, quiz bank, 10-day instructor agenda, troubleshooting playbook, and appendices A–G.

Deliverables in /curriculum:

- Exhausted-or-Nauseous-Full-Curriculum-Pack.md — editable source
- Exhausted-or-Nauseous-Full-Curriculum-Pack.pdf — print/share version
- generate_curriculum_pdf.py — regenerate PDF after edits

Use with the Cursor lesson-plan canvas for navigation; use this pack for depth, labs, and shipping.